RUNTIME NOTES · AAMAD_TARGET_RUNTIME=CLAUDE-AGENT-SDK · 30 AUG 2026

Claude Agent SDK in NovaMart

Every place this codebase touches `@anthropic-ai/claude-agent-sdk`, and why each option is set the way it is. The SDK is reached through one function call — what surrounds that call is the whole design.

SDK VERSION	IMPORT SITES	QUERY() CALLS	OPTIONS SET
0.3.241	5 files	1	17
AGENT DEFINITIONS	**HOOK EVENTS**	**MCP TOOLS**	**BUILT-INS DENIED**
5 + coordinator	4	9, read-only	17

The surface actually used

Five files import from the SDK, and only one of them imports a value. Everything else takes types only, so the module graph is never evaluated on the default path.

Every `@anthropic-ai/claude-agent-sdk` import in the repository.

FILE	IMPORTED	KIND	WHAT IT DOES HERE
`engines/sdk.ts:201`	`query`	value, dynamic	The single entry point. One call runs one customer turn.
`tools.ts:16`	`createSdkMcpServer, tool`	value	Builds the in-process MCP server with the nine read tools.
`agents.ts:19`	`AgentDefinition`	type	Shapes the five specialists handed to `options.agents`.
`hooks.ts:16`	`HookCallbackMatcher, HookInput, HookJSONOutput`	type	Types the four lifecycle callbacks and their return shapes.

FILE	IMPORTED	KIND	WHAT IT DOES HERE
stubs.ts:23	AgentDefinition	type	Deferred Sprint 2 agents. Inert — never registered, never imported by a turn.

There is no external MCP process, no SDK CLI invocation of our own, and no second `query()` anywhere — including in tests, which drive the engine seam rather than the SDK.

How the SDK is reached at all

The default engine is `deterministic`: keyless, code-composed, no SDK. The SDK engine is opt-in through `CHAT_ENGINE=sdk`, and it is loaded by dynamic import so an unconfigured or broken SDK cannot take the default path down.

```
// server/runtime/engines/select.ts
export async function loadEngine(id) {
  if (id === "deterministic")
    return { ok: true, engine: deterministicEngine };

  try {
    const mod = await import("./sdk");
    return { ok: true, engine: mod.sdkEngine };
  } catch (err) {
    return { ok: false,
             code: "sdk_engine_unavailable", ... };
  }
}
```

WHY DYNAMIC

On the deterministic path the SDK module graph is never evaluated: no CLI subprocess is located, no key is read, no network stack is touched.

The worst case for a misconfigured SDK is **one clean `error` frame** on a request that explicitly asked for it — never a crash, and never a broken demo.

```
// server/runtime/config.ts — preflight
const key = process.env.ANTHROPIC_API_KEY?.trim();
const model = process.env.MODEL_ID?.trim();
// both required; MODEL_ID is NOT defaulted
```

WHY NO DEFAULT MODEL

A silently chosen model makes the Audit line a guess. Missing either value produces a documented `error` frame naming the variable, then `done{escalated}`.

The one call, option by option

Seventeen options are set on `query()`. Nothing below is left to an SDK default — the adapter rule requires explicit budgets, and several of these exist because a default was wrong for this system.

```
const run = query({
  prompt: userPrompt,
  options: {
    model: preflight.model,
    systemPrompt,
    agents,
```

IDENTITY

`systemPrompt` is a plain string built by `coordinatorPrompt()` — not a preset, and not extended from one. The coordinator is a support router, so none of the SDK's coding-assistant framing applies.

History rides in the **user prompt as text**, not through session resume, so what the model saw is exactly what the transcript in SQLite holds.

```
    abortController,
    maxTurns: budgets.maxModelTurns,   // 12
    thinking,                          // adaptive
    effort: budgets.effort,            // low
    env: {
      ...process.env,
      CLAUDE_CODE_MAX_OUTPUT_TOKENS: "4096",
    },
```

BUDGETS

`maxThinkingTokens` is deprecated and model-dependent — on Opus 4.6 any non-zero value acts as a plain on/off switch, so the old `MAX_THINKING_TOKENS=1024` never capped anything. `thinking` plus `effort` replaced it.

Output tokens are set through the environment because that is the control the SDK reads.

```
    allowedTools: [...allowedTools, "Agent"],
    disallowedTools: [...FORBIDDEN_BUILTIN_TOOLS],
    mcpServers: { novamart: toolServer },
    hooks,
```

LEAST PRIVILEGE

Both lists are computed from the tool registry rather than written out, so they cannot drift from the per-agent allowlists or from the invariant test that guards them.

Seventeen built-ins are denied by name — `Bash`, `Write`, `Edit`, `Read`, `WebFetch`, `WebSearch` and the rest. The crew has no filesystem and no outbound network.

```
    canUseTool: async (toolName, toolInput) => {
      if (isMoneyToolName(toolName))
        return { behavior: "deny",
                 message: "No money tool exists." };

      if (DELEGATION_TOOL_ALIASES.includes(toolName))
        return { behavior: "allow",
                 updatedInput: {
                   ...toolInput,
                   run_in_background: false } };

      return { behavior: "allow" };
    },
```

TWO JOBS

Money denial. Layer 5 of five, sharing no code with the `PreToolUse` hook that is layer 4. Twenty-five name patterns — `refund`, `charge`, `cancel`, `payout`, `void` … — are refused before anything runs.

Synchronous delegation. SDK 0.3.x launches subagents in the *background* by default: `Agent` returns `{ status: "async_launched" }` and the coordinator answers without ever seeing the specialist's result. For a support crew that is a grounding failure, so the flag is forced here. A prompt asking for it is not a control.

`updatedInput` *replaces* the tool input, so it is omitted rather than sent as `{}` when there is nothing to change.

```
    forwardSubagentText: false,
    includePartialMessages: streamMode === "live",
    persistSession: false,
    cwd: process.cwd(),
  },
});
```

ONE VOICE, NO MEMORY

Specialist text never reaches the stream loop. That is the first of two independent mechanisms; the second is a `parent_tool_use_id === null` filter on everything the loop reads.

`persistSession: false` keeps the SessionStore the single source of truth. Splitting conversation state between our SQLite file and the SDK's own would make a divergence invisible until it produced a wrong answer.

Agents — `AgentDefinition` entries, not prompts

The coordinator is the main agent. The five specialists are `AgentDefinition` entries in `options.agents`, reached through the built-in `Agent` tool, which only the coordinator holds.

```
// server/runtime/agents.ts — one of five
"order-specialist": {
  description:
    "Order and line-item facts: status, dates, " +
    "totals, items. Cannot cancel, refund, or " +
    "change an order.",
  tools: [...orderTools],
  disallowedTools: SPECIALIST_DISALLOWED,
  prompt: [ ... ].join("\n"),
}
```

WHAT EACH FIELD CARRIES

`description` is routing copy — it is what the coordinator reads when choosing. `tools` comes from `AGENT_TOOL_ALLOWLIST`, so an agent can only state facts its own tools returned.

Delegation is structural. Specialists never receive the `Agent` tool, so a specialist cannot re-route even under prompt injection — the capability is absent, not refused.

Temporal safety runs through this layer: the definitions are built by `buildAgentDefinitions(temporal)`, which hands the model `{ asOf }` and nothing else. The `shiftDays` offset that maps a 2024 dataset onto today's calendar stays inside the tool layer, where no agent can read it back off.

Tools — an in-process MCP server

Nine tools, built with `tool()` and served by `createSdkMcpServer()` in the same process. No second process to start, secure, or fail to start.

```
// server/runtime/tools.ts
const getOrder = tool(
  "get_order",
  "Look up one NovaMart order by its numeric " +
    "id. Read-only. …",
  { order_id: z.number().int().positive() },
  async (args) => { ... },
);

return createSdkMcpServer({
  name: "novamart",
  version: "1.0.0",
  instructions:
    "NovaMart read-only support tools. No tool " +
    "here can move money, refund, cancel, or " +
    "charge. …",
  tools: [ getOrder, ...eight more ],
});
```

SHAPE OF A TOOL

Name, description, a **zod schema**, handler. The schema is the validation boundary — a malformed argument never reaches the data layer.

Names reach the model namespaced as `mcp__novamart__get_order`, which is also the form the allowlists and the `PreToolUse` hook match on.

Two assertions run *before* the server is built: the registered set must contain no money vocabulary, and must equal a hard-coded literal. A money tool added here is a boot failure, not a runtime surprise.

Hooks — where the rules are actually enforced

Everything the prompts ask for politely is also made true mechanically. Four events are wired; each returns `HookJSONOutput`.

Wired in `buildHooks()`, `server/runtime/hooks.ts:252–255`.

EVENT	ENFORCES	EFFECT ON THE TURN
`PreToolUse`	Money-name denial (L4), per-agent allowlist, hop-budget refusal on the delegation tool	Returns `permissionDecision: "deny"` with a reason the model reads

EVENT	ENFORCES	EFFECT ON THE TURN
`SubagentStart`	Hop accounting — a hop is an *agent transfer*, not a tool call	Increments `hops`; a spent budget forces escalation
`PostToolUse`	Outcome ledger for the escalation package's `tools_tried` and citations	Writes the operator trace; nothing customer-visible
`SubagentStop`	Hop close-out	Specialist text is logged for operators, never streamed

```
return {
  hookSpecificOutput: {
    hookEventName: "PreToolUse",
    permissionDecision: "deny",
    permissionDecisionReason: reason,
  },
};
```

DENIAL SHAPE

The reason is written for the model, not for a log: it names what was refused and what to do instead, which is how a denied handoff becomes an escalation rather than a stuck turn.

Two ledgers are kept, not one. A tool that returns an error never reaches `PostToolUse`, so attempts are recorded at `PreToolUse` and outcomes at `PostToolUse`. Attempts minus successes is what tells the engine the data had nothing to give.

Reading the stream

The engine iterates the async generator `query()` returns and acts on three message types. Everything else — including every subagent message — is ignored by construction.

`stream_event`	**Live mode only.** `content_block_delta` / `text_delta` on the main thread becomes a `token` frame, through a filter that holds back a possible partial control marker.

`assistant`	**Final mode:** buffered as the reply. **Live mode:** marks a paragraph boundary, because a coordinator that speaks before and after delegating used to run two sentences together mid-word.
`result`	**First one wins.** A turn can surface more than one `result`; acting on the later one overwrote the real answer and doubled the trace line. `subtype !== "success"` becomes a clean error frame plus `done{escalated}`.

```
function mainAgentText(message) {
  if (message.parent_tool_use_id != null) return "";
  ...
}
```

SINGLE VOICE, TWICE OVER

`forwardSubagentText: false` should mean specialist text never arrives. This filter assumes it might anyway. One customer voice is the product claim, so it does not rest on one flag.

Determinism and the Audit line

Every control that affects reproducibility is an environment variable resolved in one place, recorded in the Prompt Trace *before* execution, and reported in the artifact Audit.

Resolved by `resolveBudgets()` and `preflightSdkEngine()`, `server/runtime/config.ts`.

ENV VAR	DEFAULT	REACHES THE SDK AS
ANTHROPIC_API_KEY	required	credential; absent ⇒ clean error frame
MODEL_ID	required	`options.model`
MODEL_EFFORT	low	`options.effort`
MAX_THINKING_TOKENS	unset	`thinking: {type:"adaptive"}`
MAX_MODEL_TURNS	12	`options.maxTurns`
MAX_OUTPUT_TOKENS	4096	`env.CLAUDE_CODE_MAX_OUTPUT_TOKENS`

ENV VAR	DEFAULT	REACHES THE SDK AS
MAX_HOPS	4	hook budget, not an SDK option
TURN_TIMEOUT_MS	120000	route `AbortController` → `options.abortController`
SDK_STREAM_MODE	final	`options.includePartialMessages`

The Prompt Trace is written before `query()` is called, carries the rendered system and user prompts, the per-agent tool grants and the resolved budgets, and runs every record through a redactor so no key can reach disk. Trace logs land in `project-context/2.build/logs/`, one JSONL file per conversation.

What the SDK offers that this build does not use

Each of these was available and declined for a stated reason, rather than simply not reached for.

CAPABILITY	WHY NOT
Session resume / fork	Conversation state would live in two places. History is replayed as prompt text so the SQLite transcript is what the model saw.
Built-in tools	All seventeen are in `disallowedTools`. A support crew needs no shell, no filesystem, no web.
External MCP servers	ADR-07: in-process, so there is no second process to authorize or fail to start.
System-prompt presets	The coding-assistant framing does not apply. The prompt is written from scratch.
Background subagents	The 0.3.x default. Forced off in `canUseTool` — an answer composed before the specialist returns is ungrounded.
ClaudeSDKClient	A turn is request-scoped and one-shot. The bidirectional client buys nothing the SSE route does not already provide.

> **The honest limit.** The SDK path is opt-in and costs money to run, so CI and the offline demo exercise the deterministic engine instead. The SDK path is verified separately: `scripts/eval-sdk.mjs` drives a live server started with `CHAT_ENGINE=sdk`, and its 114 assertions check the SSE wire and the JSONL trace. Both engines satisfy the same DTO and `/api/health` reports which one is live — but they are only ever shown to agree on the 18 integration cases that run on both, not on arbitrary input.

`@anthropic-ai/claude-agent-sdk 0.3.241 · 1 query() call · 17 options · 5 AgentDefinition entries · 9 MCP tools · 4 hook events · 17 built-ins denied`